UNSA

UNIVERSIDAD NACIONAL DE SAN AGUSTÍN DE AREQUIPA

INFORME TÉCNICO
PROYECTO INTEGRADOR

Aplicación cloud-native en AWS: pruebas automatizadas, CI/CD, infraestructura como código, Kubernetes, autoescalado y monitoreo

Curso: Cloud Computing

Integrantes:

Johan Lizarve Mamani
Sebastian Mendoza Fernandez
Sergio Ahmed Castillo Sancho

Arequipa, noviembre de 2026

1. Resumen

El proyecto toma una aplicación de laboratorio que simula un exchange de apuestas (backend FastAPI, frontend nginx y base de datos PostgreSQL), originalmente ejecutada en un clúster Kubernetes local con *kind*, y la lleva a un ciclo de vida cloud-native completo en Amazon Web Services. Se incorporaron: pruebas automatizadas (unitarias e integración), integración continua y entrega continua con GitHub Actions, infraestructura como código con Terraform (EKS y ECR), orquestación en Kubernetes con sondas, secretos y recursos definidos, autoescalado horizontal (HPA) validable con pruebas de carga k6, y monitoreo con Prometheus y Grafana.

La propiedad de negocio que se vigila en todas las pruebas es **drift = wallets.available - SUM(transactions.amount) = 0**: el saldo de cada billetera debe coincidir siempre con la suma de su libro de movimientos. La liquidación de apuestas es idempotente gracias a una clave única (*liquidation_key*), de modo que un reintento no paga dos veces.

2. Alcance y estado de verificación

Se indica con honestidad qué fue ejecutado y verificado y qué está implementado pero pendiente de evidencia de ejecución en AWS.

Elemento	Estado	Evidencia
Pruebas unitarias (41)	Verificado	pytest local y CI en GitHub
Pruebas de integración (17, PostgreSQL real)	Verificado	pytest local (Postgres 16) y CI en GitHub
Análisis estático (ruff)	Verificado	CI en GitHub
Construcción de 3 imágenes Docker	Verificado	Job build del CI
terraform validate / plan	Verificado	plan: 15 recursos a crear (cuenta AWS Academy, us-east-1)
Compilación Kustomize (base y overlay aws)	Verificado	kubectl kustomize
Creación de EKS y ECR (terraform apply)	Verificado	Clúster chaos-lab-eks, 3 nodos, addon EBS CSI activo, 3 repos ECR
CD (GitHub Actions) y despliegue en EKS	Verificado	Run en verde: publish, deploy y smoke test; versión c25cdd7 en vivo
Caos, HPA con carga k6, Prometheus	Verificado	Resultados en la sección 8 y en docs/evidencias/
Capturas de Grafana y pipeline con test roto	Pendiente	Se muestran en la demo

3. Arquitectura

El flujo completo es el siguiente:

Etapa	Descripción
1. Desarrollo	El desarrollador trabaja en una rama y abre un Pull Request hacia main en GitHub.
2. Integración continua	GitHub Actions ejecuta ruff, pytest (unitarias e integración con servicio postgres:16) y construye las imágenes. Un test roto detiene el pipeline.

Etapa	Descripción
3. Entrega continua	Con CI verde en main, el workflow CD construye y publica las imágenes en Amazon ECR etiquetadas con el SHA corto del commit, despliega en EKS, ejecuta un smoke test y, si falla, revierte con kubectl rollout undo.
4. Ejecución	Usuario -> balanceador de AWS -> Traefik (ingress) -> backend (2 a 8 réplicas, HPA) -> PostgreSQL (StatefulSet con volumen EBS). Frontend nginx con 2 a 6 réplicas.
5. Observabilidad	Prometheus recoge /metrics del backend, kube-state-metrics y cAdvisor; Grafana muestra el dashboard propio 'Chaos Lab - Backend y HPA' (8 paneles).
6. Infraestructura	Terraform crea y destruye EKS, el grupo de nodos (3 x t3.medium), el addon EBS CSI y tres repositorios ECR.

4. Cumplimiento de los requerimientos de la rúbrica

4.1 Control de versiones, ramas y Pull Requests

- Fork propio del repositorio base (direcciones en el Anexo A).
- Rama de trabajo *feat/tests-and-ci* y Pull Request #1 hacia *main* con 5 commits y 38 archivos modificados.
- Flujo del equipo: una rama por tarea (feat/, fix/) y PR; solo se fusiona con el CI en verde.
- Documentación: README con guía del equipo, docs/RUNBOOK-AWS.md, docs/TUTORIAL-EQUIPO.md, docs/HANDOFF-IA.md y este informe.

4.2 Testing

Tipo	Cantidad	Qué cubre
Unitarias	41	Redondeo monetario half-up, rate limit, circuit breaker, sondas startup/liveness/readiness, validación de /bets, cardinalidad de métricas, endpoint /api/v1/version, generador de semillas
Integración	17	API con PostgreSQL real: apuestas BACK/LAY, saldo insuficiente, liquidación idempotente, drift = 0

Se generan reportes JUnit XML y de cobertura como artefactos del CI. La variable *REQUIRE_DB=1* hace que, si falta la base de datos en CI, las pruebas de integración fallen en vez de omitirse. El job de build declara *needs: [lint, test]*, por lo que un test fallido bloquea la construcción y publicación. Resultado verificado: 41 pruebas unitarias y 17 de integración aprobadas, y pipeline en verde en GitHub.

4.3 Contenedorización y registry

- Dockerfiles multi-stage y con usuario no root para backend, frontend y generador de carga.
- Registry: Amazon ECR (repositorios chaos-lab/backend, frontend y loadgen) con política de ciclo de vida para limpiar imágenes antiguas.
- Versionado por SHA corto del commit (no se usa latest). El argumento de build GIT_SHA se expone en GET /api/v1/version, lo que permite comprobar en vivo qué versión corre.

4.4 Infraestructura como código (Terraform)

Recurso	Detalle
Clúster EKS	chaos-lab-eks, versión 1.31, rol LabRole
Grupo de nodos	3 x t3.medium, etiqueta workload=app

Recurso	Detalle
Addon	aws-ebs-csi-driver (volúmenes de PostgreSQL)
Repositorios ECR	backend, frontend, loadgen, con lifecycle policy
Red	VPC por defecto y subredes públicas (sin NAT Gateway, para reducir costo); subredes etiquetadas para balanceadores
Proveedores	aws ~> 5.80, fijado con .terraform.lock.hcl

El estado de Terraform (*.tfstate) y el directorio .terraform están excluidos por .gitignore. Resultado: terraform validate correcto y terraform plan con 15 recursos a crear.

4.5 Integración continua

Workflow .github/workflows/ci.yml, disparado en PR y push: (1) **lint** con ruff; (2) **test** con Python 3.12 y un servicio postgres:16, que ejecuta unitarias e integración y sube JUnit y cobertura; (3) **build** de las tres imágenes mediante una matriz, solo si lint y test pasaron. Verificado en verde en el PR #1.

4.6 Entrega continua

Workflow .github/workflows/cd.yml, disparado por *workflow_run* (CI exitoso en main) y por *workflow_dispatch*. Pasos: autenticación en AWS con GitHub Secrets, build y push a ECR con el SHA corto, despliegue con scripts/deploy-aws.sh (kubectl kustomize con sustitución de tags), smoke test (scripts/smoke-test.sh) y rollback automático con *kubectl rollout undo* si el smoke test falla. Las credenciales (AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_SESSION_TOKEN) se guardan únicamente en GitHub Secrets.

4.7 Orquestación en Kubernetes

- Deployments (backend, frontend), StatefulSet (PostgreSQL), Services, Ingress con Traefik y Middlewares (rate limit y límite de peticiones simultáneas).
- Tres sondas distintas: startup, liveness (no consulta la base de datos) y readiness (sí). Así una caída de PostgreSQL saca los pods del Service sin reiniciarlos en cascada.
- Requests y limits de CPU y memoria; PodDisruptionBudget; NetworkPolicy definidas (no aplicadas por el CNI de EKS por defecto).
- Secretos: 02-secret.yaml se excluye del despliegue; deploy-aws.sh genera contraseñas aleatorias guardadas solo en el clúster.
- Overlay Kustomize para AWS (StorageClass gp2, HPA con máximo 8, límites de tasa altos para permitir la prueba de carga).
- Recuperación ante fallos: scripts/demo-chaos.sh elimina pods del backend y de PostgreSQL bajo tráfico y verifica drift = 0.

4.8 Autoescalado y pruebas de carga

El HPA escala el backend por CPU (70 %) y memoria (80 %) entre 2 y 8 réplicas en AWS, apoyado en metrics-server. La prueba de carga oficial es tests/load/k6-load.js, con los perfiles *ramp* (subida gradual) y *spike* (pico brusco). El script scripts/demo-hpa.sh registra réplicas contra tiempo en un CSV dentro de reports/. Los límites de tasa del overlay AWS se elevan porque la carga sale de una sola IP; con los valores locales se recibiría HTTP 429 antes de que la CPU suba y el HPA no se activaría.

4.9 Monitoreo

Se usa kube-prometheus-stack (Prometheus y Grafana). Un ServiceMonitor indica a Prometheus que recoja el endpoint /metrics del backend. El dashboard 'Chaos Lab - Backend y HPA' tiene 8 paneles (tasa de peticiones, latencia, errores, réplicas, CPU y memoria, entre otros). Una prueba unitaria verifica que las rutas se agrupen en las métricas para evitar cardinalidad ilimitada.

5. Selección de herramientas y justificación

Componente	Elegido	Alternativa	Justificación
Control de versiones	Git / GitHub	GitLab	Necesario para el flujo de PRs y Actions
Testing	pytest	unittest	Backend en Python; TestClient de FastAPI
Contenedores	Docker multi-stage	Podman	Dockerfiles existentes; versionado por SHA
Registry	Amazon ECR	GHCR	Integrado con EKS e IAM
CI/CD	GitHub Actions	Jenkins	Sin servidor propio; service containers
IaC	Terraform	OpenTofu	Herramienta de referencia
Orquestación	Amazon EKS	k3s	Kubernetes gestionado; permitido en Learner Lab
Monitoreo	Prometheus y Grafana	CloudWatch	La app ya expone métricas Prometheus
Carga	k6	Locust	Umbrales declarativos, scripts en JS

6. Seguridad y costos

- Ninguna credencial en el repositorio (revisión de secretos limpia). Las credenciales AWS viven en GitHub Secrets y en el archivo local de credenciales del operador.
- Contenedores sin root, límites de recursos y secretos generados aleatoriamente.
- Limitación del entorno: AWS Academy Learner Lab no permite crear roles IAM, por lo que se reutiliza LabRole y no se aplica el principio de mínimo privilegio completo.
- Costo estimado del entorno encendido: 0,25 USD por hora (EKS, 3 nodos t3.medium, volúmenes EBS y balanceador). Un clúster olvidado costaría aproximadamente 6 USD al día. El presupuesto disponible del laboratorio es reducido, por lo que el entorno se crea solo para las pruebas y la demostración.
- Destrucción: scripts/aws-down.sh desinstala los charts de Helm, elimina namespaces y PVC (que liberan balanceadores y volúmenes EBS creados por Kubernetes), ejecuta terraform destroy y lista recursos que pudieran seguir cobrando.

7. Reproducibilidad

Paso	Comando
Crear infraestructura	./scripts/aws-up.sh
Desplegar	merge a main (CD) o ./scripts/deploy-aws.sh
Verificar	./scripts/smoke-test.sh
Demostrar caos	./scripts/demo-chaos.sh
Demostrar autoescalado	k6 run -e BASE_URL=http://BALANCEADOR tests/load/k6-load.js y ./scripts/demo-hpa.sh
Destruir	./scripts/aws-down.sh

El detalle paso a paso está en docs/RUNBOOK-AWS.md y docs/TUTORIAL-EQUIPO.md del repositorio.

8. Evidencias de ejecución en AWS

Datos medidos en la ejecución del 9 de noviembre de 2026 (cuenta AWS Academy, us-east-1). Los archivos originales están en docs/evidencias/ del repositorio.

Evidencia	Resultado medido
Clúster EKS	chaos-lab-eks ACTIVE, 3 nodos t3.medium Ready
Imágenes en ECR	Etiquetadas por SHA (92af4b9, d51097b, c25cdd7 y latest)
CD en GitHub Actions	Publicación en ECR, despliegue en EKS y smoke test en verde; el paso de rollback se omite al no haber fallo
Versión en producción	GET /api/v1/version devuelve commit c25cdd7
Smoke test	/api/v1/health, /api/v1/events y / responden 200
Caos: matar un pod del backend	47 peticiones OK, 0 fallidas; el Deployment recrea el pod
Caos: matar PostgreSQL	Un 503 transitorio y recuperación en unos 25 s; reinicios del backend: 0
Consistencia del dinero	drift = 0 tras el caos y tras la carga (usuarios 1, 2, 3 y 50)
k6 (perfil ramp, 8 min, 300 VU)	218 929 peticiones, 454 req/s; p90 275,6 ms, p95 525,6 ms, media 190 ms, máximo 3,19 s; 0,86 % marcadas como fallidas (incluye respuestas 402 esperadas); umbrales p95 < 1500 ms y error < 5 % cumplidos
HPA del backend	De 2 a 8 réplicas (máximo) en aproximadamente 1 minuto desde el primer escalado (04:30:53 con CPU 112 % hasta 04:31:57); CPU pico 189 %
Prometheus	Targets del backend en estado up (ServiceMonitor)
Capturas de Grafana	Pendiente: se muestran en la demo
Gasto real del laboratorio (Used $X of $50)	Pendiente: consultar en AWS Academy

8.1 Recuperación ante fallos (requisito 4.7)

Paso	Acción	Resultado medido
1	Eliminar un pod del backend con tráfico (25 s de sondeo cada 0,2 s)	47 peticiones OK, 0 fallidas; pod nuevo en estado Running
2	Eliminar el pod kuber-postgres-0	Sondeos de /events: 000, 000, 503, 200, 200, 200; StatefulSet recuperado
3	Reinicios del backend por esa causa	0 (los dos pods con reinicios = 0)
4	Consistencia del dinero (usuario 1)	available 814,48 = ledger_sum 814,48; drift = 0,0

8.2 Prueba de carga con k6 (requisito 4.8)

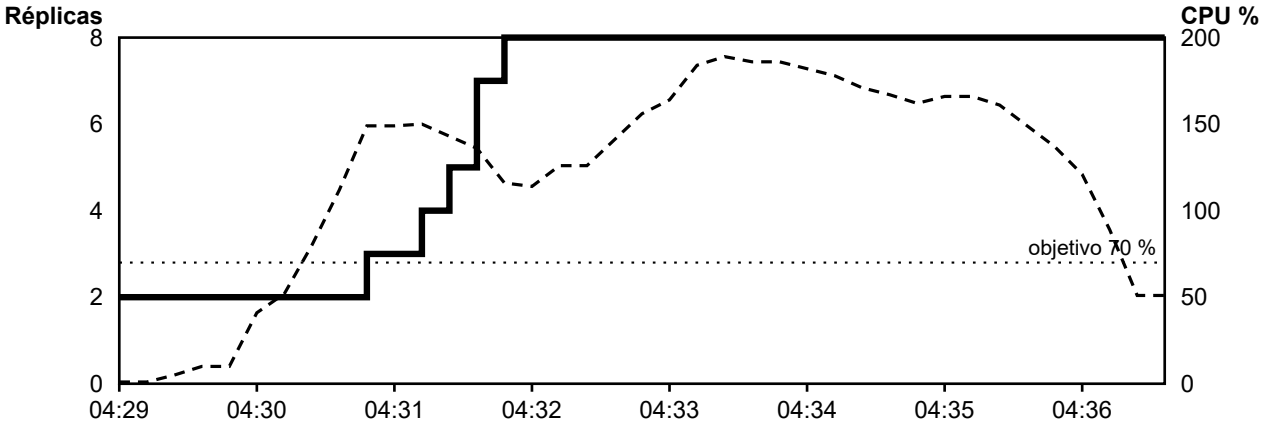
Perfil *ramp* (rampa de 1 min a 30 usuarios virtuales, 2 min a 150, 2 min a 300, 1 min sostenido y 2 min de bajada), contra el balanceador de AWS, con la aplicación en 2 a 8 réplicas.

Métrica	Valor
Duración	8 min 2 s
Usuarios virtuales máximos	300

Métrica	Valor
Peticiones HTTP totales	218 929 (454 por segundo)
Latencia media / mediana	190,3 ms / 130,0 ms
Latencia p90 / p95	275,6 ms / 525,6 ms
Latencia mínima / máxima	100,2 ms / 3 199 ms
Checks superados	100 % (218 913 de 218 913)
Peticiones marcadas como fallidas	0,86 % (1 892); incluyen respuestas 402 de apuesta con saldo insuficiente, que el check acepta
Umbral p95 < 1500 ms	Cumplido (525,6 ms)
Umbral tasa de error < 5 %	Cumplido (0,86 %)
Datos recibidos	201 MB

8.3 Comportamiento del HPA (requisito 4.8)

Serie temporal registrada cada ~13 s con scripts/demo-hpa.sh (archivo docs/evidencias/hpa.csv). Línea continua: réplicas disponibles; línea discontinua: CPU media del backend; punteada: objetivo de 70 %.



Hora	Réplicas disponibles / deseadas	CPU media
04:29:10	2 / 2	1 % (sin carga)
04:30:40	2 / 2	80 %
04:30:53	2 / 3	112 % (primer escalado)
04:31:06	3 / 4	149 %
04:31:32	4 / 5	150 %
04:31:44	5 / 7	143 %
04:31:57	7 / 8	136 %
04:32:10	8 / 8 (máximo)	116 %
04:33:53	8 / 8	189 % (pico)
04:36:53	8 / 8	89 %
04:37:18	8 / 8	51 %

- Tiempo hasta el primer escalado: unos 100 s desde el inicio de la rampa (el HPA lee métricas cada ~15 s y exige superar el 70 %).

- De 2 a 8 réplicas en 64 s (04:30:53 a 04:31:57).
- Con 8 réplicas (el máximo configurado) la CPU media siguió por encima del objetivo (116 % a 189 %): el sistema alcanzó el límite del HPA. Aun así, el p95 se mantuvo en 525,6 ms.
- La memoria se mantuvo estable (19 % a 21 %).
- Bajada de carga: al cierre del registro seguían 8 réplicas, porque el HPA aplica una ventana de estabilización de 5 minutos antes de reducir; no se registró la reducción.

8.4 Despliegue y monitoreo

- Imágenes publicadas por el CD con el SHA corto (92af4b9, d51097b, c25cdd7) y la etiqueta latest.
- Deployments backend y frontend con 2 réplicas iniciales; PostgreSQL con volumen EBS de 10 GiB (clase gp2) en estado Bound.
- Objetivos de Prometheus del backend en estado up; Grafana y el dashboard desplegados en el namespace monitoring.

Problemas encontrados y resueltos: (1) el controlador EBS CSI entraba en CrashLoopBackOff porque los pods no podían leer las credenciales del rol del nodo (límite de saltos del servicio de metadatos IMDS igual a 1); se subió a 2 y se incorporó a Terraform mediante una plantilla de lanzamiento; (2) el CD fallaba con 'Permission denied' porque los scripts no tenían el bit de ejecución en git; (3) un error de variable no definida en demo-chaos.sh.

9. Hallazgos y mejoras propuestas

- Las respuestas 500 de excepciones no controladas no se registran en las métricas porque el manejador genérico corre fuera del middleware.
- La función settle_bet contiene un return duplicado e inalcanzable.
- seed.py usa la hora actual para los eventos, por lo que el conjunto de datos no es completamente determinista.
- Mejoras: backend remoto de Terraform (S3 con bloqueo), autenticación OIDC en lugar de claves de larga duración, NetworkPolicy activas, alertas en Prometheus, análisis de vulnerabilidades de imágenes y subredes privadas con NAT en un entorno de producción.

10. Integrantes

Nombre	Curso
Johan Lizarve Mamani	Cloud Computing
Sebastian Mendoza Fernandez	Cloud Computing
Sergio Ahmed Castillo Sancho	Cloud Computing

Anexo A. Repositorios

Repositorio	Dirección
Proyecto (fork del equipo)	https://github.com/sergioandante/bet-exchange-chaos-lab-sergio
Pull Request principal	https://github.com/sergioandante/bet-exchange-chaos-lab-sergio/pull/1
Repositorio base original	https://github.com/helvetios16/bet-exchange-chaos-lab